

truesum User Manual

Exact summation of floating-point matrices

Nicholas Wilt
Archaea Software, LLC
nicholas@archaeasoftware.com

Version 1.0, 2026

Contents

1	Overview	1
2	Installation	2
2.1	Dependencies	2
2.2	Building	2
2.3	Environment	3
3	Using the library	3
3.1	A first example	3
3.2	Surveys	3
4	Reference: AccumulationMatrix	4
4.1	Construction	4
4.2	Shape	4
4.3	Accumulation	4
4.4	Threads	5
4.5	Readback	5
4.6	Column state and sizing	5
5	Reference: CudaAccumulationMatrix	6
5.1	Construction and streams	6
5.2	Reservation	6
5.3	Input memory	6
5.4	Readback and inspection	7
6	Tests and benchmarks	7
7	License	7

1 Overview

`truesum` sums floating-point matrices elementwise and exactly. An *accumulation matrix* holds one two's complement fixed-point integer per entry, wide enough that no addition rounds, so the result

is independent of the order of submission, the number of worker threads, and whether the CPU or GPU implementation performed the work. Each column shares one exponent, set at the least significant bit of any value it has received, and is stored limb-major: one contiguous allocation per 64-bit limb position, holding that limb of every row.

The library provides two classes with parallel interfaces:

- `trueSum::AccumulationMatrix` (header `trueSum/accumulation_matrix.hpp`), the CPU implementation, with a scalar kernel and an AVX-512 kernel selected at run time;
- `trueSum::CudaAccumulationMatrix` (header `trueSum/cuda_accumulation_matrix.hpp`), which keeps the accumulation matrix in device memory and accumulates with CUDA kernels.

Both accept *surveys* (`trueSum/survey.hpp`), twelve-byte per-column summaries of an input matrix that let an accumulation matrix be sized before any data arrives.

2 Installation

2.1 Dependencies

Component	Requirement
C++ compiler	C++17; tested with GCC 13 and Clang 18
CMake	3.13 or later
AVX-512 kernels (optional)	a compiler accepting <code>-mavx512f -mavx512dq -mavx512vl -mavx512bw -mavx512bf16</code>
CUDA class (optional)	CUDA Toolkit 12.x; tested with 12.9; a device of compute capability 7.0 or later
Threads	pthread, found by CMake

The library has no other dependencies. The test suite and benchmarks are dependency-free as well.

2.2 Building

From the directory containing `CMakeLists.txt`:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/test_truesum          # CPU test suite
./build/test_cuda             # CUDA test suite (if a toolkit was found)
./build/demo                  # 4x4 demonstration
```

Three options control what is built:

- `-DTRUESUM_ENABLE_AVX512=OFF` omits the AVX-512 kernels, leaving the scalar kernel. The scalar build passes the full test suite and is the configuration for a machine without AVX-512, including Apple silicon.
- `-DTRUESUM_ENABLE_CUDA=OFF` skips the CUDA toolkit detection and the `CudaAccumulationMatrix` target.
- `-DTRUESUM_BUILD_BENCH=OFF` omits the benchmark drivers.

The library builds and its tests pass on Linux (x86-64) and, in the scalar configuration, on macOS.

2.3 Environment

TRUESUM_KERNEL=scalar forces the portable kernel at run time even on a CPU with AVX-512, which is how the two kernels are cross-checked in testing. AccumulationMatrix::describe() reports which kernel is active.

3 Using the library

3.1 A first example

```
#include "truesum/accumulation_matrix.hpp"

truesum::AccumulationMatrix acc(rows, cols);

// Optional: size every column from a representative matrix and the number
// of matrices to come, so accumulation never rescales or widens.
acc.reserve_for(first_batch.data(), n_batches);

for (const auto &batch : batches) {
    acc.add_matrix_col_major(batch.data()); // column-major: preferred
    // acc.add_matrix(batch.data());       // row-major also works
}

double x = acc.to_double(i, j); // correctly rounded, once
std::string s = acc.to_exact_decimal(i, j); // every digit, never rounds
double m = acc.to_double_mean(i, j, n_batches); // mean, rounded once
```

3.2 Surveys

A Survey is a plain struct of 12 bytes:

```
struct Survey {
    int min_exponent; // lowest true-ulp exponent among nonzero values
    int max_top;      // one past the highest occupied bit position
    bool any;         // some value was nonzero
    bool nonfinite;   // some value was Inf or NaN
};

Survey survey_column(const double *values, std::size_t rows);
void survey_matrix_col_major(Survey *out, const double *b, std::size_t rows,
                             std::size_t cols, std::size_t col_stride = 0);
void survey_matrix(Survey *out, const double *b, std::size_t rows,
                   std::size_t cols, std::size_t row_stride = 0);
```

A survey describes values, not their storage: the survey of a matrix and of any exact copy of it are identical. Surveys of several matrices reduce to one reservation by taking the minimum of min_exponent and the maximum of max_top per column.

Surveys enter the library in three ways:

- at construction, AccumulationMatrix(rows, cols, surveys), where surveys[k][j] describes matrix k 's column j . The matrix is then *pre-sized*: no submission surveys, rescales or widens, and submitting more matrices than were described throws;

- `reserve_for_surveys(surveys)`, which grows the columns to cover the surveys given but keeps surveying each submission, for a caller that knows some of what is coming;
- per submission, `add_matrix_col_major(b, surveys)`, which skips the survey pass for that matrix.

A supplied survey is checked, not trusted: the accumulate kernel tests every addend against the extents it was given and throws `std::runtime_error` if a survey understated them.

4 Reference: AccumulationMatrix

All methods are in namespace `trueSum`. Row and column indices are zero-based. “Column-major” means column j begins at `b + j*col_stride` with rows contiguous; “row-major” means row i begins at `b + i*row_stride` with columns contiguous. A stride of 0 means tightly packed.

4.1 Construction

`AccumulationMatrix(rows, cols)` a dense $\text{rows} \times \text{cols}$ accumulation matrix, every column one limb wide with no exponent yet; the first submission to each column sets its scale.

`AccumulationMatrix(n, Uplo uplo)` a symmetric $n \times n$ matrix stored as one triangle, `Uplo::Lower` or `Uplo::Upper`. Inputs are assumed symmetric and only the stored triangle is read.

`AccumulationMatrix(rows, cols, surveys)`, `AccumulationMatrix(n, uplo, surveys)` pre-sized from `std::vector<std::vector<Survey>>`, one inner vector per matrix to be submitted, one `Survey` per column.

The class is movable and not copyable.

4.2 Shape

`rows()`, `cols()`, `symmetric()`, `uplo()`; `column_rows(j)` and `column_first_row(j)` give the stored length of column j and the logical row its first stored entry corresponds to (always `rows()` and 0 unless symmetric).

4.3 Accumulation

`add_matrix(b, row_stride = 0)` $A += B$, B row-major.

`add_matrix_scaled_pow2(b, log2_scale, row_stride = 0)` $A += 2^{\log_2 \text{scale}} B$, exact for any integer scale.

`add_matrix_col_major(b, surveys = nullptr, col_stride = 0)` $A += B$, B column-major; the preferred entry point, since it needs no staging. `surveys`, if given, is one `Survey` per column of B .

`add_matrix_col_major_scaled_pow2(b, log2_scale, col_stride = 0)` as above with a power-of-two scale.

`add_matrices_col_major(b, count, surveys = nullptr, col_stride = 0)` $A += B_0 + \dots + B_{\text{count}-1}$ in one pass over the accumulation matrix; `b` is an array of `count` pointers to column-major matrices and `surveys`, if given, is indexed `surveys[k][j]`. Identical results to `count` separate calls; less traffic to the accumulation matrix.

`add_column(j, v)`, `add_column_scaled_pow2(j, v, log2_scale)` $A_{:,j} += v$ for `column_rows(j)` contiguous values, so a caller can stream columns without holding a whole matrix.

`set_zero()` clears every entry but keeps each column's exponent and width, so a reused matrix does not pay again for rescaling.

Submitting an infinite or NaN value throws `std::domain_error`; on a pre-sized matrix, a value outside a column's extents, or more matrices than were declared, throws `std::runtime_error`. -0.0 accumulates as an exact zero.

4.4 Threads

`set_threads(n)` spreads accumulation over `n` workers partitioned by column; results are bit-identical to the serial ones. `threads()` reports the count. The workers persist between calls. A single accumulation matrix must not be used from more than one caller thread at a time.

4.5 Readback

`to_double(i, j)` entry (i, j) correctly rounded to nearest, ties to even, over the full range; $\pm\infty$ on overflow, and rounded once into the denormal range.

`to_double(&residual, i, j)` the same, also storing the exact difference between the stored value and the returned one, itself correctly rounded; the pair is a two-term expansion of about 106 bits.

`to_matrix(out, row_stride = 0)`, `to_matrix_with_residual(out, residual, row_stride = 0)` bulk forms of the above, writing a row-major matrix.

`to_double_mean(i, j, count)`, `to_double_mean(&residual, i, j, count)`, `to_matrix_mean(out, count, r)` the stored sum divided by `count`, correctly rounded as a quotient. `count` is a `std::uint64_t`; zero throws `std::domain_error`.

`to_exact_decimal(i, j)` the stored value as a decimal string, every digit exact.

`is_exactly_representable(i, j)`, `is_zero(i, j)` predicates on the stored value.

`entry_limbs(i, j)` the entry's limbs, least significant first, as `std::vector<limbs::limb_t>`; for testing and interoperation.

4.6 Column state and sizing

`column_exponent(j)`, `column_limbs(j)`, `column_bit_width(j)` the column's exponent (significance of bit 0 as a power of two), its limb count, and that count times 64.

`reserve_column(j, exponent, bits)` sets column `j`'s exponent (or rescales it down to `exponent` if already set) and widens it to hold `bits` bits.

`reserve_for(b, count = 1, row_stride = 0)` sizes every column from a representative row-major matrix and the number of matrices to come, so that accumulating that many similar matrices never rescales or widens. Purely an optimization; results are identical without it.

`reserve_for_surveys(surveys)` the same from surveys, without pre-sizing's promise (submissions are still surveyed).

`memory_bytes()` bytes allocated for limb storage.

`describe()` a human-readable per-column report of exponent and width, and the active kernel.

5 Reference: CudaAccumulationMatrix

The CUDA class mirrors the CPU class where the two share semantics, with these differences.

5.1 Construction and streams

Every constructor takes a trailing `CUstream_st *stream = nullptr` (a `cudaStream_t`). If given, every kernel launch, stream-ordered allocation and copy the class issues goes onto that stream; if not, the class creates a stream and destroys it with the matrix. `stream()` returns the stream in use. All submissions are asynchronous with respect to the host: they return when the work is queued. `synchronize()` waits for it. To wait for one submission only, record a CUDA event on `stream()` after it and wait on the event.

5.2 Reservation

A device column must be sized before it is accumulated into, by one of `reserve_column(j, exponent, bits)`, `reserve_for(b, count, col_stride)` (surveying a host matrix on the CPU), `reserve_for_device(b, count, col_stride)` (surveying a device-resident matrix with a kernel), `reserve_for_surveys(surveys)`, `reserve_like(cpu)` (copying the column state of an `AccumulationMatrix`), or a constructor taking surveys. A column may be reserved once; after that the class adapts on its own, rescaling or widening a column when a submission's survey falls outside it. Both adjustments drain the stream, so a reservation that covers the data is what keeps the device pipeline full.

5.3 Input memory

`add_matrix_col_major(b, surveys = nullptr, col_stride = 0)` *b* must be page-locked, device-mapped host memory: `cudaHostAlloc` with `cudaHostAllocMapped`, or `cudaHostRegister` with `cudaHostRegisterMapped`. Ordinary pageable memory is rejected with `std::invalid_argument`. The kernel streams the matrix over the bus, so the buffer must not be modified until the submission has completed.

`acquire_input()` returns one of two mapped host buffers the class keeps in rotation, sized for one matrix, blocking until the kernel that last read it has finished. A producer that fills the returned buffer and submits it with `add_matrix_col_major` never waits on the stream as a whole.

`add_matrix_col_major_device(b, surveys = nullptr, col_stride = 0)`, `add_matrices_col_major_device(b, surveys, count)` input already resident in device memory, singly or batched.

`survey_matrix_col_major_device(out, b, rows, cols, col_stride, stream)` (in `cuda_accumulation_matrix`) surveys a device-resident matrix with a kernel on the given stream, writing `cols` surveys to `out`, which may be device memory or mapped host memory. It allocates nothing and is asynchronous.

5.4 Readback and inspection

`column_exponent(j)`, `column_limbs(j)`, `entry_limbs(i, j)`, `download_column(j)` (all limbs of one column), `column_occupancy(j)` (the highest limb any entry of the column has reached), `column_contradictions(j)` (the flags raised by supplied surveys that understated the data) and `memory_bytes()`. Correctly rounded readback to `double` is done by downloading limbs and rounding on the host; see the test suite for the pattern.

6 Tests and benchmarks

`test_truesum` (CPU) and `test_cuda` (GPU) are dependency-free assertion runners; each prints a count of checks and failures and exits nonzero on any failure. Neither takes input. The suites cover exact decomposition of random bit patterns and every special value, catastrophic cancellation, correct rounding at ties including the denormal boundary, order independence over random permutations, agreement of every entry point and every kernel, and the means.

The benchmark drivers replicate the tables in the accompanying article and take no interactive input:

`bench_table_cpu [threads]` Table 1, CPU rows.

`bench_table_gpu` Table 1, GPU rows.

`bench_adjust_cpu [rows] [cols] [steps]` Table 2 and the widen/rescale costs.

`bench_adjust_gpu [rows] [cols] [steps]` the device-side counterpart, including the allocation timing.

`bench/replicate.sh <build-dir>` runs all four and writes their output to `bench/output/`; the outputs recorded on the machine used for the article are in `bench/expected/`, with the hardware identified in each file's first line. Timings are hardware-dependent; the structural claims (widen cost flat in width, rescale cost growing with it) are what a replication should confirm.

7 License

`truesum` is distributed under the BSD 3-Clause License; the text is in `Doc/LICENSE`.